

How the app is built and released

A start-to-finish guide for a developer who has never seen this repo. It explains **what the app is made of, how a build turns source into an installable app, and how a release reaches Google Play and TestFlight**. Section 11 explains how to copy the same setup into another project.

How to read it: sections 1-5 are concepts (read once). Sections 6-8 are the day-to-day how-to. Section 9 is a troubleshooting table. Sections 10-11 explain the *why* and how to reuse it. For the exact release to-do list, see `RELEASE_CHECKLIST.md`.

Contents: 1 What the app and "presets" are - 2 Glossary - 3 Repository map - 4 How a build works - 5 Keys, secrets and signatures - 6 Local development - 7 The CI/CD pipeline - 8 Cutting a release - 9 Troubleshooting - 10 Why it is built this way - 11 Reusing this in another project - 12 File index

1. What the app is, and what "presets" are

The app (working name ScanFit, to be renamed). Indian exam and government portals reject uploads that break their rules: "signature must be JPEG, 10-20 KB, 140x60 pixels". Candidates photograph their face, signature, thumb and certificates with a phone, and the file is the wrong size, so the portal says no. The app takes the phone photo and produces a file that passes. Everything happens on the device: nothing the user captures is uploaded anywhere.

A preset is one exam's upload rules, written as data. One JSON file per exam, kept in `spec/presets/exams/<category>/<id>.json`. For each document the exam asks for (photo, signature, left thumb, handwritten declaration, ID proof, certificates...) it records the allowed file formats, the size window in KB, and the pixel dimensions. It also records where the numbers came from and how sure we are. A shortened real example (`spec/presets/exams/banking/ibps_po.json`):

```
{
  "id": "ibps_po",
  "name": "IBPS PO / MT",
  "category": "banking",
  "documents": [
    { "type": "signature", "required": true, "formats": ["jpg", "jpeg"],
      "size_kb": { "min": 10, "max": 20, "target": 16 },
      "dimensions": { "mode": "preferred", "width": 140, "height": 60 } }
  ],
  "sources": [ { "url": "https://...", "kind": "secondary" } ],
  "confidence": "high",
  "last_verified": "2026-09-30"
}
```

There are **55 presets in 10 categories** (banking 10, entrance 12, SSC 8, railway 6, UPSC 6, insurance 4, defence 3, teaching 3, state PSC 2, regulator 1).

Why rules are data, not code. (a) Android and iOS must behave identically, and two copies of the numbers would drift apart. One set of files, read by both apps, cannot. (b) Fixing a wrong rule means editing a JSON file, not changing code in two languages. (c) It is a project rule (`CLAUDE.md` rule 5): never hard-code an exam rule in app code.

The presets bundle. At build time all 55 files are merged into one file, `spec/dist/presets.json`, which also carries the category list, the "popular exams" list, a disclaimer, a `schema_version` (file format) and a `presets_version` (the data's own version; it only goes up). Both apps embed this bundle, so they work offline on first launch.

What the apps do with it. The *fit engine* resizes and re-compresses an image until it satisfies a preset's rules. The *match engine* answers "does this existing file satisfy exam X?". The home screen lists and searches the exams.

Why the bundle is signed. A signature proves the bundle was produced by us and not altered. It matters for two reasons:

1. It catches build mistakes: a stale or wrong bundle cannot be shipped by accident.
2. A later phase lets the app download newer presets from a server, so rules can be fixed without a new app release. If that server were ever compromised, the app must not trust whatever it receives. It only accepts a bundle whose signature verifies with the public key built into the app.

Nothing else in this document is specific to exams. Everything about signing keys, store uploads and CI is the standard way to ship a mobile app. The *signed presets* idea is the only unusual part (section 11 treats it as optional).

2. Glossary

Term	Plain meaning
Preset	One exam's upload rules as a JSON file.
Bundle	All presets merged into <code>spec/dist/presets.json</code> .
Signature (<code>.sig</code>)	A short proof, made with a secret key, that the bundle has not changed.
Private key / seed	The secret half of the signing key pair. Whoever holds it can sign. Never committed.
Public key	The shareable half. It can only <i>check</i> a signature. It is committed and built into the apps.
Dev key / prod key	Two pairs. The dev private part is public on purpose and only for local work. The prod pair is the real one.
Spec	The <code>spec/</code> folder: the shared contract (presets, algorithms, strings, design tokens, test fixtures) that both apps follow.
Generated file	Code produced by a script from the spec (strings, theme, analytics). Committed, and CI fails if it drifts.
Debug / Release	The two build configurations. Debug is for development, Release is what ships. They use different keys.
Upload key (Android)	The key that proves to Google Play that an upload came from us.
AAB / APK	Android app formats. Play requires an AAB. An APK is installable directly.
R8 / mapping.txt	Android's code shrinker, and the file that translates shrunken crash traces back to real names.
Archive / IPA / TestFlight	iOS: the built app bundle, the installable file, and Apple's beta distribution.
XcodeGen	A tool that generates the Xcode project from <code>ios/project.yml</code> . The project file is not stored in git.
CI / GitHub Actions	Computers GitHub starts to build and test every push.
Tag	A git label (<code>android-v0.1.0-rc3</code>) on a commit. Pushing one starts a release.
Secret / environment gate	Encrypted values stored in GitHub, and a manual "approve this job" checkpoint.
rc / pre-release / final	<code>-rcN</code> tags are test builds. A tag without a suffix is a final release.

3. Repository map

```

android/    Kotlin + Jetpack Compose app. Gradle, many modules, build logic in android/build-logic/
ios/        Swift + SwiftUI app. Xcode project generated by XcodeGen from project.yml. Code lives in loca
spec/       The shared contract that BOTH apps follow
  presets/exams/<category>/<id>.json    the 55 presets (humans edit these)
  presets/VERSION                      integer, bump it with every preset change
  schema/                              JSON Schema every preset must satisfy
  signing/                             dev_public_key.b64, prod_public_key.b64 (public keys only)
  fixtures/                             test images + cases.json: the tests both apps must pass
  strings/ tokens/                     UI text (English, Hindi) and design tokens
  tools/                                Python scripts that build, sign, check and generate (section 4)
  dist/                                GENERATED bundle + signature. Git-ignored
docs/      Product, architecture, roadmap, testing, release
web/        Static site and the future presets CDN folder
.github/workflows/ spec.yml, android.yml, ios.yml

```

What is committed and what is not:

Kind	Examples	In git?
Source of truth	presets, schema, strings, tokens, app code	yes
Generated, committed	strings.xml , Localizable.xcstrings , Strings.swift , theme files, analytics events	yes (CI fails if stale)
Generated, not committed	spec/dist/ , ios/ScanFit.xcodeproj , ios/ScanFit/Info.plist , all build/ folders	no
Public keys	spec/signing/*_public_key.b64	yes
Secrets	presets private seed, Android keystore, keystore.properties , Apple .p8 key, Local.xcconfig	never (git-ignored)

4. How a build works

4.1 The data flow (applies to both apps)

```

spec/presets/exams/**/*.json          55 files, edited by people
|
| spec/tools/build_presets.py
|   1. validate each file against spec/schema
|   2. extra rules (target inside the min-max window, sources exist, dates sane...)
|   3. merge into one bundle, add version + categories + popular list
|   4. sign the exact bytes with an Ed25519 private key
v
spec/dist/presets.json + presets.json.sig      (generated, git-ignored)
|
|-- Android: Gradle task EmbedPresetsTask --> verifies the signature, then copies into the
'-- iOS:      build phase embed_presets.sh --> verifies the signature, then copies into the
                                                    (both also copy the matching PUBLIC key into t
v
App launch: the loader checks the signature again with that public key, then reads the exams.

```

Two ideas to hold on to:

1. **Python only produces the bundle.** It runs when presets change, never inside an app build. Gradle and Xcode do not run Python.
2. **The signature is checked three times:** in the script that builds the bundle, by Gradle/Xcode before embedding, and by the app at launch. A mistake has to get past all three.

`spec/tools/build_all.sh` runs the whole producer side in one command. By default it signs with the **dev** key (local work). `--release` signs with the **real** seed (from `PRESETS_SIGNING_KEY`) and also fails if any Hindi string is still machine-drafted; `--allow-todo-hi` lifts that check for pre-release builds.

4.2 Debug versus Release

	Debug	Release
Purpose	development, tests, simulators	what goes to the stores
Presets are signed with	the dev key	the prod key
Public key built into the app	<code>dev_public_key.b64</code>	<code>prod_public_key.b64</code>
Builds refuse...	nothing special	the dev key, even if copied into the prod slot
Code shrinking	off	on (Android R8)

So `spec/dist` must be signed to match the configuration you build. This is why local release builds need the real seed, and why you reset to the dev bundle afterwards (section 6).

4.3 Android build

- Gradle with convention plugins in `android/build-logic/` (shared rules), modules `core:*` (engines, model, presets...), `feature:*` (screens) and `app` (assembly). Versions: AGP 9.4.1, Gradle 9.6.1, Kotlin 2.4.20, `compileSdk` 37, `minSdk` 26, `targetSdk` 36. Android Studio **Quail 4 (2026.1.4) or newer** is needed to sync the project.
- The embed plugin (`scanfit.android.embed-presets`) adds one task per variant. `EmbedPresetsTask` verifies the signature with the JDK's own Ed25519 support (no Python), refuses the dev key in a release variant, then writes the bundle into the app's assets.
- `ReleaseConfig.kt` adds release signing and versioning (section 5 and 7).
- Release builds run R8 (shrinks and obfuscates), then produce an **AAB** (for Play) and an **APK** (for direct install).

4.4 iOS build

- `ios/project.yml` is the single description of the Xcode project. `xcodegen generate` writes `ScanFit.xcodeproj` (not committed).
- Code lives in two local Swift packages: `Core` (models, imaging, match, inspect, presets, design system...) and `Features` (Home, Kit, Settings screens). Swift 6 strict concurrency.
- `ios/Config/Debug.xcconfig` and `Release.xcconfig` set per-configuration build settings. Each optionally includes `Local.xcconfig` (git-ignored) where *you* put your Apple Team ID.
- Version and build number are build settings (`MARKETING_VERSION` , `CURRENT_PROJECT_VERSION`) that flow into `Info.plist` .
- The build phase **Embed signed presets** runs `Scripts/embed_presets.sh` , which picks the dev or prod public key by configuration and calls `Scripts/verify_presets.swift` (Apple's CryptoKit) to verify the bundle, then copies everything into the app.
- `Scripts/archive.sh` wraps the release steps: generate project, archive, check the archive, export, optionally upload.

5. Keys, secrets and signatures

There are **three separate signatures** in a release. Mixing them up is the most common source of confusion.

#	Signature	What it proves	Secret part	Who checks it
1	Presets signature	The rules bundle is ours and unmodified	presets seed (PRESETS_SIGNING_KEY)	our own app, with the built-in public key
2	Android upload signature	An upload to Play came from us	upload keystore .jks + passwords	Google Play
3	iOS code signature	Apple has certified this build and this developer	distribution certificate + provisioning profile, held by Apple ("cloud signing")	Apple, at upload and install time

Inventory of every key and secret

Item	Where it lives	If it is lost
Presets dev key	in the repo, on purpose (derived from a public string). Local use only	nothing to lose; never ship it
Presets prod seed	your password manager + GitHub secret PRESETS_SIGNING_KEY	rotate: generate a new pair, replace the secret and prod_public_key.b64 , ship a new build. Old builds keep their old key
Presets prod public key	spec/signing/prod_public_key.b64 (committed)	it is public; the file is in git
Android upload keystore	~/keys/*.jks outside the repo + GitHub secrets (4)	Google can reset the upload key because Play App Signing holds the real app key
Android app signing key	held by Google (Play App Signing)	not your responsibility
Apple Team ID	ios/Config/Local.xcconfig + secret APPLE_TEAM_ID	public information
Apple API key (.p8 , Key ID, Issuer ID)	password manager + 3 GitHub secrets; needs the Admin role	revoke and generate a new one

GitHub secrets used by the workflows: PRESETS_SIGNING_KEY , ANDROID_UPLOAD_KEYSTORE_BASE64 , ANDROID_UPLOAD_STORE_PASSWORD , ANDROID_UPLOAD_KEY_ALIAS , ANDROID_UPLOAD_KEY_PASSWORD , APPLE_TEAM_ID , ASC_KEY_ID , ASC_ISSUER_ID , ASC_KEY_P8_BASE64 .

Rules the build enforces so a mistake cannot ship:

- A release build refuses to embed the dev key, even if someone copies it into the prod slot.
- A release bundle (bundleRelease) refuses to build unsigned. It never falls back to the debug key.

- A bundle that does not verify against the key about to be embedded stops the build.

6. Local development

6.1 One-time setup

Tools: Xcode (recent, Swift 6), Android Studio **Quail 4 or newer**, a JDK 17+, `brew install xcodegen swiftlint`, Python 3.

Python is needed **only to produce** `spec/dist` (and for the key and preflight tools). macOS Python is "externally managed", so use a virtual environment (`.venv/` is git-ignored):

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r spec/tools/requirements.txt
spec/tools/build_all.sh          # validates, bundles and DEV-signs the presets; regenerates strings/token
```

Android also needs `android/local.properties` containing `sdk.dir=/path/to/Android/sdk` (git-ignored).

6.2 iOS: run the app from Xcode (no Python involved)

```
cd ios && xcodegen generate      # creates ScanFit.xcodeproj (re-run after editing project.yml)
open ScanFit.xcodeproj
```

Pick the `ScanFit` scheme and a simulator, then Run. The simulator needs no Apple account or Team ID. For a real iPhone, put `DEVELOPMENT_TEAM = <your 10-character Team ID>` in `ios/Config/Local.xcconfig` (copy `Local.xcconfig.example`).

Tests: `swift test --package-path Packages/Core` (engines and conformance tests), `swift test --package-path Packages/Features`, and `xcodebuild -scheme ScanFit -destination 'platform=iOS Simulator,name=iPhone 17'` test for the app. `swiftlint --strict` must pass.

6.3 Android

Open `android/` in Android Studio, or use the command line from `android/` :

```
./gradlew :app:assembleDebug
./gradlew spotlessCheck detekt testDebugUnitTest lint
```

6.4 Changing presets

1. Edit the JSON file(s) in `spec/presets/exams/` .
2. **Bump** `spec/presets/VERSION` . Apps refuse a bundle with a lower version than the one they already have.
3. Run `spec/tools/build_all.sh` . It validates every file and rebuilds the bundle.
4. Run both apps' tests (`spec/README.md` has the full procedure).

6.5 The dev/prod switch (the one awkward thing)

`spec/dist` is signed with **one** key at a time, and the build configuration must match it (section 4.2):

- Debug builds need the **dev**-signed bundle: `spec/tools/build_all.sh` .
- A local Release/Archive build needs the **prod**-signed bundle: paste the seed silently, then sign.

```
read -rs "PRESETS_SIGNING_KEY?Paste the seed, then press Enter: " && export PRESETS_SIGNING_KEY
PYTHON=.venv/bin/python spec/tools/build_all.sh --release --allow-todo-hi
```

Afterwards run `spec/tools/build_all.sh` again to return to the dev bundle. Real releases do not need this: CI signs with the secret.

6.6 Optional: a signed release build on your own machine

- **Android:** copy `android/keystore.properties.example` to `keystore.properties` and fill it in (git-ignored), sign the bundle with the prod seed as above, then `cd android && ./gradlew :app:bundleRelease -Pscanfit.versionName=1.0.0 -Pscanfit.versionCode=1` .
- **iOS:** `cd ios && DEVELOPMENT_TEAM=<id> Scripts/archive.sh --version 1.0.0 --build 1` (add `--upload` to send to TestFlight). `--no-sign` compiles and verifies the archive without any Apple account.

`python3 spec/tools/release_preflight.py` lists everything still missing for a release. It is read-only and never prints a secret.

7. The CI/CD pipeline

CI means GitHub starts a fresh computer, checks out the code and runs the steps in `.github/workflows/` . There are three workflows.

Workflow	Starts on	Jobs
<code>spec.yml</code>	changes under <code>spec/</code> or <code>web/</code>	<code>validate</code> , then <code>publish-presets</code> (on main only)
<code>android.yml</code>	changes under <code>android/</code> or <code>spec/</code> , and tags <code>android-v*</code>	<code>build</code> (always), <code>release</code> (tags only)
<code>ios.yml</code>	changes under <code>ios/</code> or <code>spec/</code> , and tags <code>ios-v*</code>	<code>build</code> (always), <code>release</code> (tags only)

7.1 build jobs: run on every push and pull request

Android build (Linux): create the dev-signed presets bundle; format check (Spotless), static analysis (detekt), unit and conformance tests, lint; the 90 % coverage gate for the engine modules; assemble the debug APK; then the **release rehearsal**.

iOS build (macOS): create the dev-signed bundle; print the Swift/Xcode versions; test the signature verifier against the shared signing vector; SwiftLint (strict); Core tests with coverage (the string-catalog tests are skipped here, see section 9); an annotation listing any failing tests; the coverage gate; Features tests; app tests plus the string-catalog tests on an **iPhone SE (3rd gen)** simulator (the low-end profile); then the **release rehearsal**.

The **release rehearsal** is the key safety net. Release-only problems (R8 breaking the app, release lint, the Release archive, the embed checks) would otherwise appear only on release day. So every push builds the release configuration, *unsigned*, using a throwaway presets key made by `spec/tools/rehearsal_key.sh`. That script refuses to run anywhere except GitHub Actions, because it overwrites `prod_public_key.b64`. No secret is used.

7.2 release jobs: run only for tags

A tag looks like `android-v1.2.3`, `android-v1.2.3-rc1`, `ios-v1.2.3` or `ios-v1.2.3-rc1`.

- **Version comes from the tag.** Android: `versionName` = the tag text after `-v`, `versionCode` = the CI run number (Google Play needs it to rise on every upload). iOS: `version` = digits only (`1.2.3` ; Apple rejects suffixes), `build number` = the run number.
- **Pre-release vs final.** A suffix (`-rc1`, `-beta1`) means pre-release and allows machine-drafted Hindi (currently 102 strings still marked `TODO_HI:`). A tag without a suffix is final and fails until a native speaker has reviewed the Hindi.
- **An approval gate.** The job pauses on an *environment* (`android-release`, `ios-release`). Open the run and choose Review deployments, then Approve. This stops a mistyped tag from shipping.

Android release steps:

1. Read version and channel from the tag.
2. Decode the upload keystore from `ANDROID_UPLOAD_KEYSTORE_BASE64` into a temporary file (secrets are text, not files).
3. Build the presets bundle signed with the real seed (`build_all.sh --release`).
4. Preflight: fail if any release input is missing (`release_preflight.py --platform android`).
5. Gradle builds the AAB and APK with R8 and signs them with the upload key, passing the version through `-Pscanfit.* flags`.
6. Verify the AAB is signed (`jarsigner`).
7. Save the AAB, APK and `mapping.txt` as downloadable artifacts. **Uploading to Play is manual** for the first release: Google opens its publishing API only after one manual upload.

iOS release steps:

1. Read the version and channel from the tag.
2. Build the presets bundle signed with the real seed.
3. Preflight (`--platform ios`).
4. Decode the App Store Connect API key from `ASC_KEY_P8_BASE64`. The step accepts base64 or raw text and checks it is a real private key before Xcode sees it.
5. Run `ios/Scripts/archive.sh --version V --build N --upload`, which: generates the Xcode project; archives the Release build (the embed phase verifies the bundle); checks the archive (dev key not embedded, debug symbols present); writes `ExportOptions.plist`; exports with `-allowProvisioningUpdates` and the API key, so Apple creates the distribution certificate and App Store profile itself (**cloud signing**, which is why the key needs the Admin role); and uploads straight to TestFlight.
6. Delete the key file, and keep the debug symbols as an artifact.

7.3 spec.yml

`validate` checks the spec: bundle and dev-sign, generated files up to date (drift check), tool tests, fixture generator, and the upload-test page tests. `publish-presets` runs on `main` after approval, signs with the real seed, verifies against the committed public key and stages the files for the future CDN folder. It is also the cheapest proof that the GitHub secret and the committed public key still match.

7.4 Safety nets built into the pipeline

Safety net	What it prevents
Release rehearsal on every push	release-only breakage found on release day
Preflight (<code>release_preflight.py</code>)	starting a release with a missing key, id, version mismatch or machine Hindi
Dev key refused in Release	shipping a bundle anyone could have forged
<code>bundleRelease</code> refuses an unsigned build	uploading something Google would reject
Shared signing vector, run on both platforms	the two verifiers drifting apart
Failing-test annotation	needing to open a long log to see which test failed
Approval gates on <code>*-release</code> and <code>presets-production</code>	accidental releases

8. Cutting a release (runbook)

1. **CI is green** on `main` (all three workflows). Presets changed since the last release? `spec/presets/VERSION` was bumped.
2. Optional: `python3 spec/tools/release_preflight.py` to see what is missing.
3. **Choose the version.** `rc` while testing, final when Hindi is reviewed. `android/gradle.properties` and `ios/project.yml` hold the default version, which must match; the tag overrides it in CI.
4. **Tag and push** (at most three tags in one push, or GitHub starts no workflows):

```
git tag android-v0.1.0-rc4 && git tag ios-v0.1.0-rc4
git push origin android-v0.1.0-rc4 ios-v0.1.0-rc4
```

5. Open each run and approve the pending gate.
6. **Android:** when it finishes, download `android-release-<version>` from the run page, install the APK on a phone to smoke-test it, then upload the AAB in Play Console under Internal testing.
7. **iOS:** the build appears in App Store Connect under TestFlight once Apple has processed it (usually minutes). Add testers.
8. **If a release job fails:** a tag is pinned to one commit. Fix the problem, push the fix, then create a *new* tag (`rc5`). "Re-run jobs" reuses the old commit, so it only helps for secret or configuration problems, not code.

One-time store setup (accounts, app records, App Store Connect API key, Play App Signing) is in `RELEASE_CHECKLIST.md` .

9. Troubleshooting

Message or symptom	Cause	Fix
<code>spec/dist/presets.json</code> does not verify against <code>prod_public_key.b64</code>	<code>spec/dist</code> is signed with a different key than this build embeds	Rebuild it: <code>build_all.sh --release</code> with the real seed, or let CI do it
Debug build says it does not verify against the dev key	<code>spec/dist</code> is still prod-signed from a release build	<code>spec/tools/build_all.sh (dev)</code>
<code>prod_public_key.b64</code> is the public DEV key	the dev key was copied into the prod slot	restore the real key from git
Release builds need a <code>python3</code> with <code>cryptography</code>	an old <code>embed_presets.sh</code> ; the current one has no Python	check the file mentions <code>verify_presets.swift</code> ; an editor may have overwritten it
<code>ModuleNotFoundError: cryptography OR externally-managed-environment</code>	system Python lacks packages, and <code>pip</code> refuses to install into it	use the <code>.venv</code> from section 6.1
<code>keyPathInvalid</code> (iOS release)	<code>ASC_KEY_P8_BASE64</code> is not a valid <code>.p8</code>	<code>base64 -i AuthKey_<ID>.p8</code> , set the secret again
Cloud signing permission error / No profiles for '<bundle id>'	the App Store Connect API key is not Admin	create an Admin key, update the three <code>ASC_*</code> secrets
Upload fails with no app record	the app does not exist in App Store Connect yet	create it with the same bundle id
String tests return keys like "tab.home" under <code>swift test</code>	SwiftPM's native build copies the <code>.xcstrings</code> catalog without compiling it	skip <code>DesignSystemTests</code> in <code>swift test</code> ; they run through <code>xcodebuild</code>
unable to type-check this expression in reasonable time on CI only	the runner's Swift compiler is older than yours	split the expression into smaller <code>let s</code>
Android Studio: "incompatible version of the Android Gradle plugin"	Studio is older than AGP 9.4	update to Quail 4 (2026.1.4) or newer
<code>implementation class Scanfit_*Plugin</code> not found	VS Code's Gradle import clobbered <code>build-logic/convention/build</code>	<code>java.import.gradle.enabled=false</code> in VS Code; see the README troubleshooting
A tag was pushed and nothing ran	more than three tags in one push, or the name does not start with <code>android-v / ios-v</code>	push tags separately, check the name
Play rejects the upload: version code already used	version code is the CI run number and must rise	tag again; never reuse a build

10. Why it is built this way

- **One spec, two apps.** Behaviour is defined in `spec/` and tested by shared fixtures; the apps do not copy each other's code. Parity comes from the contract and the shared test vectors.
- **Data is signed, and checked at every hop.** A wrong bundle has to get past the build script, the Gradle/Xcode step and the app.
- **Fail loudly, never fall back.** No release build can silently use the debug signing key or the dev presets key.
- **Dev and prod are separate on purpose.** The dev key is public so anyone can run the project; the real key exists only in a password manager and GitHub.
- **Versions come from the tag.** Nobody edits a version number by hand and forgets.
- **Rehearse the release on every push.** The release path is exercised continuously with throwaway credentials.
- **Python stays out of app builds.** Builds depend only on Xcode and Gradle, so an IDE or a new laptop needs no extra tooling.
- **Humans approve releases.** Tags start the work; environment gates make a person confirm it.

11. Reusing this in another project

11.1 What is portable

Part	Reusable for any Android or iOS app?
Tag-triggered release pipeline, signing, versions from the tag, approval gates	Yes, copy all of it
Rehearsal, preflight, safety nets	Yes
Shared <code>spec/</code> with parity fixtures across two native apps	Only if you build two native apps from one contract
Signed data bundle (this project's presets)	Optional. Needed only if you ship data that must be provably authentic or updated over the air

If your app has no such data, skip the signed-bundle parts (11.4) and you still get a complete release pipeline.

11.2 Build it in this order (each step is usable on its own)

1. **A signed release build on your own machine.** Android: an upload keystore and a Gradle signing config. iOS: an archive that exports.
2. **One source of version numbers**, overridable from outside (a Gradle property, Xcode build settings).
3. **CI build job on every push:** tests, lint, a debug build.
4. **Android release job on a tag:** secrets, an approval gate, signed AAB as an artifact.
5. **iOS release job on a tag:** App Store Connect API key (Admin), cloud signing, upload to TestFlight.
6. **Safety nets:** release rehearsal on every push, a preflight script, refuse-to-fall-back rules.
7. **(Optional) signed data bundle.**

11.3 Templates

Android: signing and versions (Kotlin DSL, simplified from `ReleaseConfig.kt`). With no credentials the release stays unsigned; it never falls back to the debug key.

```

val props = java.util.Properties().apply {
    rootProject.file("keystore.properties").takeIf { it.exists() }?.inputStream()?.use { load(it) }
}
fun cred(env: String, key: String): String? = System.getenv(env) ?: props.getProperty(key)

android {
    defaultConfig {
        versionName = providers.gradleProperty("app.versionName").getOrNull("0.1.0")
        versionCode = providers.gradleProperty("app.versionCode").getOrNull("1").toInt()
    }
    val storePath = cred("UPLOAD_STORE_FILE", "storeFile")
    if (storePath != null) {
        signingConfigs.create("upload") {
            storeFile = rootProject.file(storePath)
            storePassword = cred("UPLOAD_STORE_PASSWORD", "storePassword")
            keyAlias = cred("UPLOAD_KEY_ALIAS", "keyAlias")
            keyPassword = cred("UPLOAD_KEY_PASSWORD", "keyPassword")
        }
    }
    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            signingConfigs.findByName("upload")?.let { signingConfig = it }
        }
    }
}

```

Create the key once, keep it outside the repo, and turn it into a secret:

```

keytool -genkeypair -v -keystore ~/keys/app-upload.jks -alias app-upload -keyalg RSA -keysize 4096 -vali
base64 -i ~/keys/app-upload.jks | pbcopy          # paste into the GitHub secret ANDROID_UPLOAD_KEystore_BAS

```

Android: release workflow skeleton

```

name: android
on:
  push:
    tags: ['android-v*']
jobs:
  release:
    runs-on: ubuntu-latest
    environment: android-release          # create in Settings > Environments; add yourself as reviewer
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21' }
      - uses: gradle/actions/setup-gradle@v4
      - id: tag
        run: echo "version=${GITHUB_REF_NAME#android-v}" >> "$GITHUB_OUTPUT"
      - name: Keystore from secret
        env: { KS: '${{ secrets.ANDROID_UPLOAD_KEYSTORE_BASE64 }}' }
        run: |
          echo "$KS" | base64 --decode > "$RUNNER_TEMP/upload.jks"
          echo "UPLOAD_STORE_FILE=$RUNNER_TEMP/upload.jks" >> "$GITHUB_ENV"
      - name: Signed bundle
        env:
          UPLOAD_STORE_PASSWORD: '${{ secrets.ANDROID_UPLOAD_STORE_PASSWORD }}'
          UPLOAD_KEY_ALIAS: '${{ secrets.ANDROID_UPLOAD_KEY_ALIAS }}'
          UPLOAD_KEY_PASSWORD: '${{ secrets.ANDROID_UPLOAD_KEY_PASSWORD }}'
        run: >
          ./gradlew :app:bundleRelease
          -Papp.versionName=${{ steps.tag.outputs.version }} -Papp.versionCode=${{ github.run_number }}
      - uses: actions/upload-artifact@v4
        with: { name: release, path: app/build/outputs/bundle/release/*.aab }

```

iOS: the pieces

- Show version and build from build settings, otherwise XcodeGen/Xcode may freeze a literal 1.0 in Info.plist and TestFlight will reject the second upload: `CFBundleShortVersionString = $(MARKETING_VERSION)`, `CFBundleVersion = $(CURRENT_PROJECT_VERSION)`.
- Keep the Team ID out of git: `Release.xcconfig` contains `DEVELOPMENT_TEAM =` then `#include?` "Local.xcconfig" (git-ignored file).
- Archive, then export and upload. Both commands take the same API-key flags:

```

AUTH=(-allowProvisioningUpdates -authenticationKeyPath "$KEY" -authenticationKeyID "$KEY_ID" -authenticationKeyContents "$KEY_CONTENTS")
xcodebuild archive -scheme App -configuration Release -destination 'generic/platform=iOS' -archivePath build/Archive
MARKETING_VERSION="$V" CURRENT_PROJECT_VERSION="$N" DEVELOPMENT_TEAM="$TEAM" "${AUTH[@]}"
xcodebuild -exportArchive -archivePath build/App.xcarchive -exportPath build/export \
  -exportOptionsPlist ExportOptions.plist "${AUTH[@]}"

```

`ExportOptions.plist` needs: `method = app-store-connect`, `destination = upload` (or `export` to only get the IPA), `teamID`, `signingStyle = automatic`, `uploadSymbols = true`. See `ios/Scripts/archive.sh` for a complete, working script.

- Workflow: `runs-on: macos-latest`, `environment: ios-release`, `steps`: decode the .p8 secret to a file, run the script, delete the file.

Store and account setup (once per app)

- Apple: developer account; register the explicit bundle id; create the app record in App Store Connect; create an API key under Users and Access, Integrations with the **Admin** role (a lower role fails with "Cloud signing permission error"); store the Team ID, Key ID, Issuer ID and the `.p8` (base64) as secrets.
- Google: create the app in Play Console; keep **Play App Signing** on (your keystore is then only the *upload* key and Google can reset it); upload the first AAB by hand.

11.4 Optional: a signed data bundle

Use it only if shipped data must be authentic (rules, prices, config) or will later be downloaded. Otherwise ship a plain JSON asset.

1. Generate an Ed25519 key pair. Commit the **public** key, put the **private** one in a password manager and a CI secret.
2. Sign the exact bytes of the file: `signature = Ed25519.sign(private, bytes)` , stored as base64 next to it.
3. Verify at **build time** (Android: `Signature.getInstance("Ed25519")` in a Gradle task; iOS: `CryptoKit.Curve25519.Signing.PublicKey.isValidSignature`) and again **at app launch** with the key compiled into the app.
4. Keep a **dev** pair and a **prod** pair. The release build refuses the dev public key.
5. Add a monotonic data version so an old bundle cannot replace a newer one.
6. Write a shared test vector (valid, tampered, wrong key, malformed signature, empty) and run it on both platforms; see `spec/fixtures/signing/` and `ios/Scripts/test_verify_presets.sh` .
7. Plan for losing the private key: it can only be rotated by shipping a new app build with the new public key.

11.5 Pitfalls we hit, so you can skip them

1. **Xcode build phases do not see your shell PATH or virtual environment.** Do not call Python or other tools from a build phase; use what ships with Xcode (here a Swift script).
2. **XcodeGen writes a literal version into Info.plist** unless you reference `$(MARKETING_VERSION)` and `$(CURRENT_PROJECT_VERSION)` .
3. **SwiftPM's native build does not compile .xcstrings catalogs.** Tests that read localized text fail under `swift test` ; run them through `xcodebuild` .
4. **The CI compiler is older than your IDE.** Print the toolchain versions in CI, and split long mixed-type expressions.
5. **Cloud signing needs an Admin API key.**
6. **Store files in secrets as base64, and validate them** before use, so a wrong paste gives a readable error.
7. **A tag points at one commit.** Fix and push, then cut a new tag; do not expect a re-run to pick up new code.
8. **Guard helper scripts that overwrite real files** (like the throwaway-key script) with something only CI sets (`GITHUB_ACTIONS`), never a variable you could type by hand.
9. **An editor with a stale tab can overwrite a file** changed on disk. Close the tab or revert it before committing.
10. **GitHub starts no workflows when more than three tags are pushed together.**
11. **Play needs a version code that always rises.** Use the CI run number.
12. **Install the linters locally** (SwiftLint here) so the first run is not on CI.
13. **Do not trust a build you have not exercised in the release configuration.** That is what the per-push rehearsal is for.

12. File index

Path	Purpose
spec/presets/exams/**/*.json , spec/presets/VERSION	the presets and their data version
spec/schema/exam.schema.json	what a valid preset looks like
spec/tools/build_presets.py	validate, bundle, sign, verify, generate keys
spec/tools/build_all.sh	run the whole producer side (dev or --release)
spec/tools/release_preflight.py	read-only list of what blocks a release
spec/tools/rehearsal_key.sh	CI-only throwaway key for per-push release rehearsal
spec/signing/*_public_key.b64	public keys built into the apps
spec/fixtures/signing/	the shared signing test vector
android/build-logic/convention/.../ReleaseConfig.kt	Android release signing and versions
android/build-logic/convention/.../EmbedPresetsTask.kt	verify and embed the bundle (Android)
android/gradle.properties , android/keystore.properties.example	default version; credentials template
ios/project.yml	the Xcode project definition
ios/Config/*.xcconfig , Local.xcconfig.example	per-configuration settings; your Team ID
ios/Scripts/embed_presets.sh	Xcode build phase: choose key, verify, embed
ios/Scripts/verify_presets.swift	Ed25519 signature check (CryptoKit)
ios/Scripts/archive.sh	archive, check, export, upload
ios/Scripts/test_verify_presets.sh	verifier against the signing vector
.github/workflows/{spec,android,ios}.yml	the pipelines
docs/RELEASE_CHECKLIST.md	the exact one-time and per-release to-do list
docs/ARCHITECTURE.md	app architecture, modules, data and security design